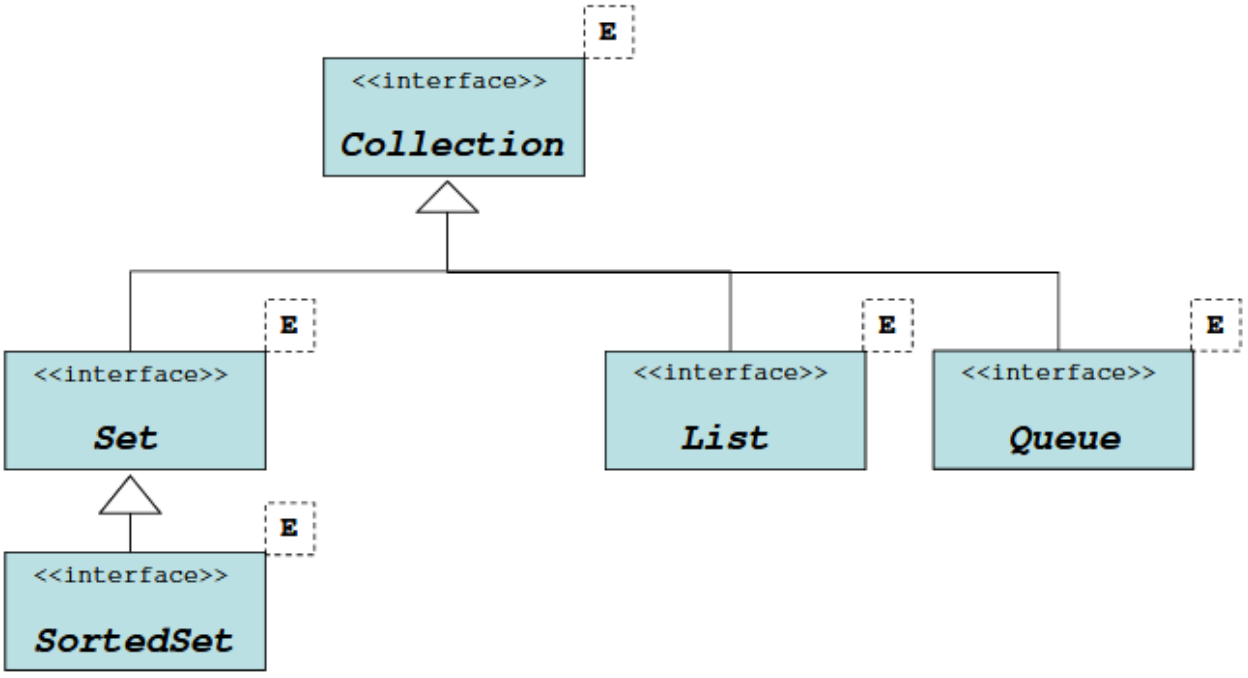
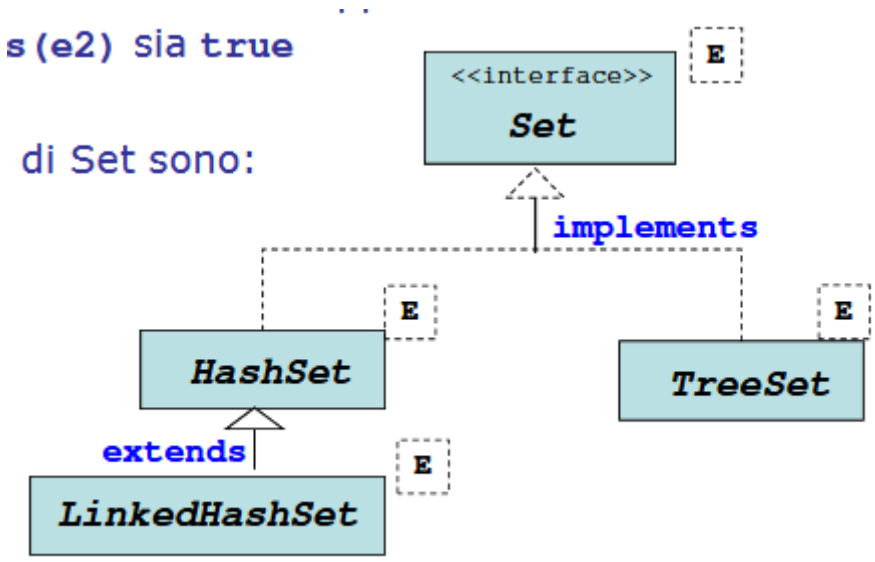
Lezione 12 parte 2 - L'interfaccia Set

L'interfaccia `Set` in Java

Questo tipo di interfaccia estende l'interfaccia `Collection`,



- quindi eredita tutti i metodi generici per aggiungere, rimuovere, verificare la presenza di elementi e ottenere un iteratore.
- Le principali implementazioni di `Set` sono:
  - `HashSet` – **Basato su una tabella hash, non mantiene ordine tra gli elementi.**
  - `LinkedHashSet` – **Mantiene l'ordine di inserimento degli elementi.**
  - `TreeSet` – **Mantiene gli elementi ordinati secondo l'ordine naturale o un comparatore definito.**



In particolare l'interfaccia `Set` rappresenta una **collezione di tipo matematico**, cioè un insieme di elementi **unici**, senza duplicati.

Ogni implementazione di `Set` **non ammette elementi duplicati**:

- formalmente, non possono esistere due elementi `e1` ed `e2` tali che

```java
e1.equals(e2) == true
```

all'interno dello stesso insieme
In sintesi, un `Set` è utile quando **vogliamo garantire l'unicità degli elementi all'interno di una collezione, evitando duplicati in maniera automatica.**

Class `HashSet` in Java

La classe `HashSet` è una delle implementazioni più comuni dell'interfaccia `Set` e serve per:

- **gestire insiemi di oggetti unici, cioè senza duplicati.**
- **Tuttavia, a differenza di altre implementazioni come `LinkedHashSet` o `TreeSet`, non mantiene alcun ordine tra gli elementi.**
- **Struttura interna:**
  - `HashSet` utilizza una **tabella hash** per memorizzare gli elementi.
  - In pratica, **internamente è rappresentato come un array di liste (chiamate *bucket*).**
    - **Ogni elemento viene inserito in un bucket specifico calcolato tramite il metodo `hashCode()`.**

- **Come evita i duplicati:**
  Per garantire l'unicità degli elementi, gli oggetti contenuti nel `HashSet` dovrebbero ridefinire:
  1. `equals(Object obj)` – **per stabilire quando due oggetti sono "uguali"**
  2. `hashCode()` – **per determinare in quale bucket deve essere collocato l'oggetto**

> **Importante: il metodo `hashCode()` deve rispettare il contratto di Java:**
>
> `Se x.equals(y) è true allora x.hashCode() == y.hashCode()`
>
> **Questo assicura che oggetti considerati uguali finiscano nello stesso bucket.**

- **Flusso di inserimento:**
  1. **Viene chiamato `hashCode()` dell'oggetto per determinare il bucket.**
  2. **Se nel bucket ci sono altri oggetti con lo stesso hash code, viene invocato `equals()` per verificare che non ci siano duplicati.**
- **Configurazione:**
  È possibile specificare, tramite i costruttori di `HashSet`, i **l numero iniziale di bucket e il fattore di carico (load factor) – cioè la soglia di riempimento che determina quando la tabella hash deve essere ridimensionata.**
  - Valori predefiniti: 16 bucket, load factor 0.75

> **Analogia: `HashSet` come uno schedario**
>
> Per capire come funziona un `HashSet`, possiamo immaginarlo come un vecchio **schedario a cassettini**.
> Ogni cassettino corrisponde a quello che in Java chiamiamo un **bucket:**
>
> - **ovvero una piccola "sezione" interna della struttura.**
>
> **Quando inseriamo un oggetto nell' `HashSet`, un algoritmo calcola automaticamente in quale cassettino collocarlo, usando il codice hash dell'oggetto.**
> In questo modo l'utente non deve preoccuparsi di scegliere l'ordine:
> l' `HashSet` gestisce tutto internamente.

Se proviamo a inserire un elemento già presente nello stesso cassettino, il sistema lo ignora, garantendo così che **non ci siano duplicati**.

Questa analogia ci permette di capire due aspetti fondamentali di un `HashSet`:

1. **Unicità: ogni elemento può comparire una sola volta, proprio come un fascicolo nello schedario non può essere duplicato.**
2. **Ordine non garantito: l'ordine degli elementi nello schedario non dipende da chi li inserisce; allo stesso modo, in un `HashSet` non possiamo prevedere l'ordine in cui gli oggetti vengono restituiti, perché la struttura li distribuisce automaticamente nei bucket.**

In pratica, l' `HashSet` ci offre un modo semplice e veloce per **conservare elementi unici** senza preoccuparci di come sono ordinati, lasciando che sia la struttura stessa a gestire tutto.

> In sintesi, `HashSet` **è ideale quando vogliamo assicurare l'unicità degli elementi con un accesso rapido (O(1) in media) ma non ci interessa mantenere l'ordine di inserimento o un ordinamento naturale degli elementi.**

Il metodo `hashCode()` e i suo ruolo nei `Set`

Abbiamo già accennato al metodo `hashCode()` nella lezione precedente e al suo legame con il metodo `equals()`.
Ora approfondiamo perché questo metodo è fondamentale per le implementazioni concrete dell'interfaccia `Set`.

Ogni oggetto in Java possiede questo metodo, ereditato dalla classe `Object`:

- **restituisce un numero intero rappresentativo dell'oggetto stesso.**
- **Questo numero viene utilizzato internamente per distribuire gli oggetti nei bucket** delle strutture hash, come nell'analogia dello schedario: **ogni oggetto finisce in un "casellario" determinato dal suo hashCode, senza che l'utente debba preoccuparsi dell'ordine.**

Proprietà chiave del "contratto" di `hashCode()`

Per garantire che le strutture hash funzionino correttamente, il metodo `hashCode()` deve rispettare alcune regole fondamentali:

1. **Consistenza:**
   - **Ogni volta che viene invocato su un oggetto, il metodo deve restituire lo stesso valore, a meno che lo stato interno dell'oggetto non cambi.**
2. **Compatibilità con `equals()`:**
   - **Se `x.equals(y)` restituisce `true`, allora necessariamente `x.hashCode() == y.hashCode()`.**
   - **Se `x.equals(y)` restituisce `false`, non è obbligatorio che i due hashCode siano diversi, ma hashCode diversi garantiscono che gli oggetti non siano uguali.**

Quindi, in parole semplici:

- **oggetti considerati uguali da `equals()` devono finire nello stesso bucket, mentre oggetti diversi possono occasionalmente condividere uno stesso bucket in caso di collisione, anche se questo riduce l'efficienza.**

> **Nota: Se si sovrascrive il metodo `equals()`, è sempre necessario sovrascrivere anche `hashCode()`, altrimenti la struttura hash non funzionerà correttamente.**

Classi che utilizzano le chiavi hash

Le strutture dati più comuni che sfruttano il metodo `hashCode()` sono:

- `HashSet` e `LinkedHashSet`
- `HashMap` e `LinkedHashMap`
- `Hashtable`

Un buon metodo `hashCode()` **distribuisce uniformemente gli oggetti tra i bucket, evitando che troppi elementi si concentrino nello stesso bucket e rallentino le operazioni.**
Ovviamente, è possibile che il metodo restituisca lo stesso valore per tutti gli oggetti, ma in questo caso la struttura hash perderebbe completamente la sua efficienza.

Esempio pratico con `HashSet<String>`

e `String` in Java forniscono già una corretta ridefinizione dei metodi `equals()` e `hashCode()`.

- **Questo significa che possono essere utilizzate direttamente all'interno di un `HashSet` senza necessità di interventi aggiuntivi da parte del programmatore.**

Java

```java
HashSet<String> ha = new HashSet<String>();

ha.add("serpente");
ha.add("ape");
ha.add("farfalla");
ha.add("farfalla"); // NON viene aggiunto, duplicato!
ha.add("furetto");
ha.add("gattino");

for (String entry : ha) {
    System.out.println("elemento: " + entry);
}
```

Analizziamo cosa accade:

- Quando inseriamo `"farfalla"` la seconda volta, l'oggetto non viene aggiunto.
  - Questo perché il `HashSet` calcola prima l' `hashCode()` della stringa per individuare il bucket e, trovando nello stesso bucket un elemento con uguale hash, invoca anche `equals()` per verificare l'uguaglianza logica.
  - Poiché le due stringhe risultano uguali, il duplicato viene scartato.
- L'**ordine di navigazione non è prevedibile**.
  - L'iterazione non segue l'ordine di inserimento, ma quello determinato dalla distribuzione interna nei bucket.
  - Per questo motivo, l'output può presentare gli elementi in un ordine apparentemente casuale.

Per avere un'idea di come sono distribuiti gli oggetti, possiamo guardare ai loro hashCode:

nginx

```
serpente → 1373560042
gattino  → -188685104
farfalla → 927458191
ape      → 96790
```

```
5   furetto  → -505888915
```

Notiamo che ogni stringa produce un valore intero differente.
Questo valore viene utilizzato per determinare il bucket di destinazione.

Non è necessario conoscere l'algoritmo con cui viene calcolato il bucket:

- **la struttura gestisce internamente la distribuzione,**
- **garantisce l'unicità degli elementi,**
- **e consente operazioni di inserimento e ricerca mediamente molto rapide.**

In sintesi, questo esempio mostra concretamente il legame tra `hashCode()` ed `equals()` e chiarisce perché il rispetto del loro "contratto" sia essenziale per il corretto funzionamento delle strutture basate su hashing.

La classe `LinkedHashSet`

La classe `LinkedHashSet` è un **sottotipo di** `HashSet` che mantiene tutte le caratteristiche tipiche delle strutture basate su hashing (unicità degli elementi, utilizzo dei bucket, efficienza nelle operazioni), ma introduce una differenza fondamentale:

- **preserva l'ordine di inserimento degli elementi.**

Questo significa che, a differenza di `HashSet`, quando si attraversa la collezione tramite un iteratore, **gli elementi vengono restituiti nello stesso ordine in cui sono stati inseriti.**

Struttura interna

Dal punto di vista strutturale, `LinkedHashSet` utilizza ancora:

- **una tabella hash (array di bucket),**
- **il metodo** `hashCode()` **per determinare il bucket,**
- **il metodo** `equals()` **per verificare eventuali duplicati.**

Anche in questo caso, gli oggetti contenuti nell'insieme dovrebbero ridefinire correttamente:

- `boolean equals(Object obj)` → **per stabilire l'uguaglianza logica**
- `int hashCode()` → **per la gestione dei bucket**

Rimane valido il contratto fondamentale:

> **Se** `x.equals(y)` **è** `true`**, allora** `x.hashCode() == y.hashCode()`**.**

Durante l'inserimento:

1. **viene invocato prima** `hashCode()` **per individuare il bucket;**
2. **se nel bucket esistono elementi con lo stesso hash, viene invocato** `equals()` **per verificare l'eventuale duplicato.**

Anche per `LinkedHashSet` è possibile specificare:

- **numero iniziale di bucket**
- **fattore di carico (load factor)**

I valori di default restano **16** e **0.75**, come in `HashSet`.

`LinkedHashSet` come uno schedario con registro cronologico

Se il `HashSet` era paragonabile a uno **schedario con cassettini (bucket)** in cui i fascicoli vengono inseriti in base a una regola interna (l'hashCode), il `LinkedHashSet` p**uò essere visto come lo stesso schedario, ma con in più un registro che tiene traccia dell'ordine di inserimento dei fascicoli.**

Immaginiamo quindi:

- uno **schedario con cassettini**, dove ogni fascicolo viene collocato automaticamente nel cassettino stabilito dal suo codice (hashCode);
- accanto allo schedario, un **registro cronologico** in cui viene annotato l'ordine con cui i fascicoli vengono inseriti.

Cosa cambia rispetto al caso precedente?

Nel `HashSet`:

- **i fascicoli vengono distribuiti nei cassettini,**
- **quando li consultiamo, li troviamo nell'ordine determinato internamente dallo schedario (non prevedibile).**

Nel `LinkedHashSet`:

- **i fascicoli vengono sempre distribuiti nei cassettini tramite hashCode,**

- **ma il registro cronologico permette di recuperarli nell'esatto ordine in cui sono stati inseriti.**

Importante:

l'ordine di inserimento non influenza la posizione nei bucket, ma influisce sull'ordine di navigazione quando percorriamo l'insieme.

Perché mantiene l'ordine?

La differenza rispetto a `HashSet` è che `LinkedHashSet`, oltre alla tabella hash, mantiene internamente anche una **struttura collegata tra gli elementi**, che consente di ricordare l'ordine di inserimento.

Questo comporta:

- **un leggero overhead in memoria rispetto a `HashSet`,**
- **ma una navigazione prevedibile.**

Esempio pratico:

Java

```java
LinkedHashSet<String> lha = new LinkedHashSet<String>();

lha.add("serpente");
lha.add("ape");
lha.add("farfalla");
lha.add("farfalla"); // NON viene aggiunto!
lha.add("furetto");
lha.add("gattino");

for (String entry : lha) {
    System.out.println("elemento: " + entry);
}
```

Analisi del comportamento:

- **Il duplicato `"farfalla"` non viene inserito, esattamente come in `HashSet`.**
- **La differenza sta nell'ordine di iterazione.**

L'iteratore mostrerà:

```nginx
serpente
ape
farfalla
furetto
gattino
```

L'ordine corrisponde esattamente alla sequenza di inserimento.
L'ultimo elemento stampato sarà quindi l'ultimo inserito.

La classe `TreeSet`

La classe `TreeSet` è un'implementazione dell'interfaccia `Set` che, a differenza di `HashSet` e `LinkedHashSet`, **mantiene gli elementi ordinati secondo un criterio di confronto.**

Non utilizza una tabella hash, **ma una struttura basata su alberi binari di ricerca bilanciati (in particolare, un albero Red-Black).**

Questo comporta una differenza concettuale importante:

- `HashSet` → **organizza per hashing (velocità, ordine non garantito)**
- `LinkedHashSet` → **hashing + ordine di inserimento**
- `TreeSet` → **ordinamento basato su confronto logico tra elementi**

Criterio di ordinamento

Affinché un `TreeSet` possa ordinare i propri elementi, è necessario che esista un **criterio di confronto**.
Questo può essere fornito in due modi:

1 Tramite `Comparable`

Quando una classe implementa l'interfaccia `Comparable<T>`, significa che **gli oggetti di quella classe sanno confrontarsi tra loro**.

```java
int compareTo(T other)
```

Come funziona il `compareTo`?

Il metodo deve restituire:

- un numero **negativo** → **se l'oggetto corrente viene prima di** `other`
- **0** → **se i due oggetti sono considerati equivalenti**
- un numero **positivo** → **se l'oggetto corrente viene dopo** `other`

Questo definisce il cosiddetto **ordine naturale**.

Esempio concettuale con `String`

e `String` implementano già `Comparable<String>`.
Quando scrivi:

```java
TreeSet<String> set = new TreeSet<>();

```

Il `TreeSet` usa automaticamente il metodo `compareTo()` delle stringhe.

Per esempio:

```java
"ant".compareTo("dog")

```

restituisce un valore negativo → quindi `"ant"` viene prima di `"dog"`.

Non devi fare nulla: l'ordine alfabetico è già definito nella classe `String`.

Quindi un `TreeSet` creato con:

```java
TreeSet<T> set = new TreeSet<>();
```

utilizzerà automaticamente il metodo `compareTo()` degli oggetti inseriti.

Quando usare il `Comparable`?

Si usa quando:

- esiste **un solo criterio naturale di ordinamento**
- vuoi che quell'ordinamento sia "intrinseco" alla classe

Ad esempio:

- numeri → ordine crescente
- stringhe → ordine alfabetico
- date → ordine cronologico

2 Tramite `Comparator`

A volte però:

- **la classe non implementa `Comparable`**
- **oppure vuoi ordinare gli oggetti in modo diverso dall'ordine naturale**

In questi casi si usa un `Comparator`.

Un `Comparator<T>` definisce il metodo:

Java

```java
int compare(T o1, T o2)
```

Qui il confronto non è definito nella classe stessa, ma in un oggetto esterno.

ⓘ Differenza concettuale

- `Comparable` → **l'oggetto sa confrontarsi da solo**
- `Comparator` → **il confronto è deciso da un oggetto esterno**

È una differenza di progettazione molto rilevante.

Esempio:

Immagina una classe `Studente` con:

- nome
- media voti

Qual è l'ordine naturale?

- Per nome?
- Per media?
- Per età?

Non è ovvio. Potresti voler ordinare in modi diversi.

In questo caso:

- NON definisci un unico `compareTo`
- Crei diversi `Comparator`:
  - uno per nome
  - uno per media
  - uno per età

E li passi al costruttore:

Java

```java
TreeSet<Studente> set = new TreeSet<>(comparatorePerMedia);
```

⚠ **Il `Comparator` non è un metodo della classe**

ma è un oggetto esterno che definisce una regola di confronto
Per comprendere meglio:

1. `Comparable` → metodo dentro la classe
   Se la classe `Studente` implementa `Comparable<Studente>`, allora il metodo `compareTo()` è definito dentro la classe stessa:

Java

```java
public class Studente implements Comparable<Studente> {

    private String nome;
    private double media;

```

```java
    @Override
    public int compareTo(Studente other) {
        return this.nome.compareTo(other.nome);
    }
}

```

Qui l'ordine è interno alla classe.
Ogni `Studente` sa confrontarsi con un altro `Studente`.
Quindi in questo caso si può istanziare il `TreeSet` così:

```java
TreeSet<Studente> set = new TreeSet<>();
```

Perché il `TreeSet` usa automaticamente `compareTo()`.

1. `Comparator` → **oggetto esterno**
   Il `Comparator` invece **non è un metodo della classe**.
   È un oggetto separato che implementa l'interfaccia `Comparator<T>`.
   Esempio:

```java
Comparator<Studente> comparatorePerMedia = new Comparator<Studente>() {
    @Override
    public int compare(Studente s1, Studente s2) {
        return Double.compare(s1.getMedia(), s2.getMedia());
    }
};

```

Oppure con lambda (più moderno):

```java
Comparator<Studente> comparatorePerMedia =
    (s1, s2) -> Double.compare(s1.getMedia(), s2.getMedia());
```

Dopodiché lo si passa al `TreeSet`:

```java
TreeSet<Studente> set = new TreeSet<>(comparatorePerMedia);
```

Ora il `TreeSet` userà quel criterio per ordinare gli studenti.

In questo caso si istanzia il `TreeSet` così:

```java
TreeSet<T> set = new TreeSet<>(comparator);
```

Il confronto verrà effettuato usando il metodo `compare()`.

ⓘ Tipi che hanno già un ordine naturale

Alcune classi, come:

- `String`
- Tipi wrapper (`Integer` , `Double` , `Character` , ecc.)

implementano già `Comparable` , quindi possiedono un **ordine naturale predefinito**.

Ad esempio:

- Le `String` → ordine alfabetico (lessicografico)
- Gli `Integer` → ordine numerico crescente

Esempio pratico con `TreeSet<String>`

```java
TreeSet<String> set = new TreeSet<String>();

set.add("dog");
set.add("ant");
set.add("horse");
set.add("gorilla");

for (String element : set) {
```

```
        System.out.println(element);
    }
```

Anche se gli elementi vengono inseriti in questo ordine:

```nginx
dog
ant
horse
gorilla
```

L'iterazione produrrà:

```nginx
ant
dog
gorilla
horse
```

Questo perché:

- il `TreeSet` **non conserva l'ordine di inserimento, ma mantiene sempre gli elementi ordinati secondo il criterio di confronto stabilito.**

> **Aspetto fondamentale: il confronto determina anche l'unicità**
>
> Nel `TreeSet`, l'unicità non dipende da `hashCode()` ed `equals()`, ma dal confronto:
>
> - Se `compareTo()` (o `compare()`) restituisce `0`,
> il `TreeSet` considera i due elementi uguali e non inserisce il duplicato.
>
> Questo è un punto molto importante:
> nel `TreeSet` l'uguaglianza è definita dal criterio di ordinamento.

`TreeSet` is-a `SortedSet`

`TreeSet` implementa l'interfaccia `SortedSet`.
Questo significa che:

- **eredita tutti i metodi di** `Set`

- **in più, mette a disposizione operazioni che sfruttano l'ordinamento interno degli elementi**

Poiché gli elementi sono sempre mantenuti ordinati, è possibile selezionare **porzioni dell'insieme** in modo efficiente.

`subSet(fromElement, toElement)`

Java

```java
SortedSet<E> subSet(E fromElement, E toElement)
```

Restituisce una **vista** del sottoinsieme che contiene tutti gli elementi:

Shell

```shell
>= fromElement
<  toElement
```

Quindi:

- l'elemento iniziale è **incluso**
- l'elemento finale è **escluso**

> ⚠ **Attenzione**
>
> Se `fromElement` è maggiore di `toElement` secondo il criterio di ordinamento, viene sollevata:
>
> Java
>
> ```java
> java.lang.IllegalArgumentException
> ```
>
> Questo perché l'intervallo non è coerente con l'ordine naturale o con il `Comparator`.

`headSet(toElement)`

Java

```java
SortedSet<E> headSet(E toElement)
```

Restituisce una vista contenente tutti gli elementi:

```java
< toElement
```

Si parte dalla “testa” (cioè dall’elemento più piccolo) fino al valore indicato, escluso.

È utile quando si vogliono tutti gli elementi “prima di” un certo valore.

`tailSet(fromElement)`

```java
SortedSet<E> tailSet(E fromElement)
```

Restituisce una vista contenente tutti gli elementi:

```shell
>= fromElement
```

Si parte dall’elemento specificato fino alla “coda” (cioè il più grande).

È utile quando si vogliono tutti gli elementi “a partire da” un certo valore.

Punto fondamentale: sono *viste*, non copie

Questo è l’aspetto più importante da comprendere.

I sottoinsiemi restituiti:

- **non sono copie indipendenti**
- **sono viste collegate all’insieme originale**

Questo significa che:

- **se modifichi il `TreeSet` originale, cambia anche il sottoinsieme**
- **se modifichi il sottoinsieme, cambia anche l’insieme originale**
  - **(purché la modifica rispetti i vincoli dell’intervallo)**

Esempio concettuale

Supponiamo di avere:

```csharp
[ant, dog, gorilla, horse, zebra]
```

Se fai:

```java
subSet("dog", "zebra")
```

Ottieni:

```csharp
[dog, gorilla, horse]
```

Se poi aggiungi `"fox"` al `TreeSet` originale:

```csharp
[ant, dog, fox, gorilla, horse, zebra]
```

Anche il `subSet` conterrà `"fox"` automaticamente, perché rientra nell'intervallo.

Questo è possibile possibile perché il `TreeSet` **è basato su una struttura ad albero ordinata.**
L'ordinamento permette di individuare in modo efficiente intervalli di elementi.

Con un `HashSet` questo non sarebbe possibile, perché non esiste un ordine.

Backed Collection

Una **Backed Collection** è una collezione che:

- **non è una copia indipendente, ma una vista collegata a un'altra collezione.**

Nel caso del `TreeSet`, i metodi:

- `subSet(...)`
- `headSet(...)`
- `tailSet(...)`

restituiscono viste backed, cioè sottoinsiemi "fusi" con l'insieme originale.

Esempio passo per passo

Java

```java
TreeSet<String> set = new TreeSet<String>();
set.add("ant");
set.add("dog");
set.add("gorilla");
set.add("horse");

SortedSet<String> subset;
subset = set.subSet("bat", "gorilla");
// intervallo: >= "bat" e < "gorilla"
```

Stato iniziale
`set` contiene (in ordine):

C#

```csharp
[ant, dog, gorilla, horse]
```

L'intervallo richiesto è:

Plain text

```
>= "bat"
<  "gorilla"
```

Quindi inizialmente `subset` contiene:

C#

```csharp
[dog]
```

Perché:

- `"ant"` < `"bat"` → escluso
- `"dog"` ≥ `"bat"` e < `"gorilla"` → incluso
- `"gorilla"` è escluso (limite superiore esclusivo)

Modifica del set principale

Java

```java
set.add("bat");
```

Ora `set` diventa:

C#

```
[ant, bat, dog, gorilla, horse]
```

Poiché `"bat"` rientra nell'intervallo, automaticamente:

Ini

```ini
subset = [bat, dog]
```

Il sottoinsieme si aggiorna da solo, perché è una vista collegata.

Modifica del subset

Java

```java
subset.add("fish");
```

"`fish`" è:

CSS

```
>= "bat"
<  "gorilla"
```

Quindi è valido.

Il nuovo stato sarà:

Ini

```ini
set = [ant, bat, dog, fish, gorilla, horse]
```

```
subset = [bat, dog, fish]
```

Anche qui la modifica si riflette su entrambe le strutture.

Inserimento fuori intervallo nel set principale

```java
set.add("zebra");
```

È fuori dall'intervallo del `subset`, ma:

- **è una modifica legittima sul** `set`
- **semplicemente non compare nel** `subset`

Quindi nessun errore.

Inserimento fuori intervallo nel subset

```java
subset.add("pig");
```

Qui si verifica un errore:

```java
java.lang.IllegalArgumentException: key out of range
```

Perché `"pig"` non rientra nell'intervallo:

```
>= "bat"
<  "gorilla"
```

Il `subset` impone il vincolo dell'intervallo.
Non puoi inserire elementi che violano il range stabilito alla creazione della vista.

In sintesi

Una Backed Collection:

- **è una vista dinamica**

- **condivide la struttura dati con l'insieme originale**
- **mantiene i vincoli di intervallo**
- **non permette inserimenti fuori range**

`TreeSet` **is-a** `NavigableSet`

`TreeSet` non implementa solo `SortedSet`, ma anche `NavigableSet`:

- **che estende** `SortedSet` **aggiungendo metodi di navigazione fine all'interno dell'insieme ordinato.**

L'idea è questa:

> Se gli elementi sono ordinati, posso chiedere "qual è il più vicino sopra?" o "qual è il più vicino sotto?" rispetto a un certo valore.

Quindi grazie a `NavigableSet`, il `TreeSet` offre metodi per muoversi nell'insieme come in una struttura ordinata navigabile.

`first()`

- **Restituisce il più piccolo elemento dell'insieme.**

Java

```java
E first()
```

`last()`

- Restituisce il **più grande elemento** dell'insieme.

Java

```java
E last()
```

`ceiling(E element)`

Restituisce:

> Il più piccolo elemento dell'insieme **maggiore o uguale** a `element` (≥)

Se esiste un elemento uguale, restituisce quello.

```java
E ceiling(E element)
```

`floor(E element)`

Restituisce:

> **Il più grande elemento dell'insieme minore o uguale a `element` (≤)**
>
> **Se esiste un elemento uguale, restituisce quello.**

```java
E floor(E element)
```

`higher(E element)`

Restituisce:

> **Il più piccolo elemento strettamente maggiore di `element` (>)**
>
> **Non include mai l'elemento stesso.**

```java
E higher(E element)
```

`lower(E element)`

Restituisce:

> **Il più grande elemento strettamente minore di `element` (<)**

```java
E lower(E element)
```

Esempio concreto

```java
TreeSet<Integer> treeset = new TreeSet<Integer>();

treeset.add(2);
treeset.add(5);
```

```
treeset.add(7);
treeset.add(33);
```

L'insieme ordinato sarà:

```
[2, 5, 7, 33]
```

Ora analizziamo:

```
treeset.floor(6);
```

Risultato: **5**

È il numero più grande ≤ 6.

```
treeset.ceiling(6);
```

Risultato: **7**

È il numero più piccolo ≥ 6.

```
treeset.higher(6);
```

Risultato: **7**

È il più piccolo > 6.

```
treeset.lower(6);
```

Risultato: **5**

È il più grande < 6.

Interfaccia `Queue`

```java
public interface Queue<E> extends Collection<E>
```

Una `Queue` rappresenta una struttura dati di tipo **FIFO** (*First In, First Out*).

Significa che:

> **Il primo elemento inserito è il primo a essere rimosso.**

È il comportamento tipico di una coda reale:

- **chi arriva prima viene servito prima.**

Operazioni tipiche di una Queue

- `add(e)` / `offer(e)` → **inserimento in coda**
- `remove()` / `poll()` → **rimozione del primo elemento**
- `element()` / `peek()` → **lettura del primo elemento senza rimozione**

La classe `PriorityQueue`

`PriorityQueue` è un'implementazione dell'interfaccia `Queue`.

Ma attenzione:

> **Non rispetta il principio FIFO.**

È una **coda con priorità**.

Funzionamento

Gli elementi non vengono rimossi nell'ordine di inserimento, ma:

> Vengono rimossi secondo un criterio di priorità.

La priorità è determinata da:

- `compareTo()` se gli oggetti implementano `Comparable`
- oppure da un `Comparator` fornito al costruttore

Struttura interna

Internamente non è una semplice lista, ma una struttura basata su un **heap** (albero binario parzialmente ordinato).

Questo garantisce che:

- **L'elemento con priorità più alta sia sempre accessibile in testa.**
- **L'inserimento e la rimozione siano efficienti**

⚠ Attenzione: iterazione ≠ ordine di priorità

Qui sta il dettaglio spesso oggetto di confusione.
Se si usa:

```java
for (E e : priorityQueue)
```

oppure un `Iterator`,

> **L'ordine degli elementi non è garantito né FIFO né ordinato per priorità.**

L'iteratore non restituisce gli elementi ordinati.

Se invece si usa:

```java
poll()
```

Gli elementi vengono rimossi:

- **In ordine di priorità.**

Metodi principali di `PriorityQueue`

Oltre ai metodi ereditati da `Collection`, una `PriorityQueue` mette a disposizione le operazioni tipiche delle code.

`peek()`

- **Restituisce l'elemento in testa (cioè quello con priorità più alta)**
- **Non lo rimuove**
- **Se la coda è vuota, restituisce `null`**
  È concettualmente equivalente a "leggere il primo elemento".

```java
E peek()
```

`offer(E e)`

- **Inserisce un elemento nella coda**
- **Rispetta il criterio di priorità**
- **Funziona praticamente come `add(e)` (nelle implementazioni standard non c'è differenza sostanziale)**

Non significa "mettere in fondo" come in una coda FIFO classica:

- **l'elemento verrà posizionato secondo la logica dell'heap interno.**

```java
boolean offer(E e)
```

`poll()`

- **Rimuove e restituisce l'elemento in testa**
- **La testa è sempre l'elemento con priorità più alta**
- **Se la coda è vuota, restituisce `null`**

È il metodo che fa emergere chiaramente l'ordinamento per priorità.

```java
E poll()
```

Esempio con oggetti `Museo`

Supponiamo che la classe `Museo` implementi `Comparable<Museo>` ordinando per **location in ordine alfabetico**.

```java
```

```java
PriorityQueue<Museo> qq = new PriorityQueue<Museo>();

qq.offer(new Museo("Louvre","Paris"));
qq.offer(new Museo("Uffizi","Firenze"));
qq.offer(new Museo("Capodimonte","Napoli"));
qq.offer(new Museo("El Prado","Madrid"));
```

Le location inserite sono:

```nginx
Paris
Firenze
Napoli
Madrid
```

Ma la priorità è alfabetica sulla location:

```nginx
Firenze
Madrid
Napoli
Paris
```

Uso corretto: rimozione con `poll()`

```java
System.out.println(qq.poll().getLocation());
```

Le chiamate successive a `poll()` produrranno:

```nginx
Firenze
Madrid
Napoli
Paris
```

Perché ogni volta viene rimosso l'elemento con priorità più alta (cioè il più piccolo in ordine alfabetico).

Attenzione: iterazione con `foreach`

Nel caso in cui si faccia:

```java
for(Museo museo : qq)
    System.out.println(museo.getLocation());
```

Il risultato **non è garantito in ordine di priorità**.
Si potrebbe ottenere.

```nginx
Firenze
Madrid
Paris
Napoli

```

oppure un'altra sequenza.

Perché succede questo?

Perché:

- `PriorityQueue` **garantisce solo che la testa sia l'elemento con priorità massima**
- **Non garantisce che tutti gli elementi siano ordinati tra loro**
- **Internamente usa un heap, non una struttura completamente ordinata come** `TreeSet`

L'heap mantiene una proprietà locale (il padre è ordinato rispetto ai figli), ma non un ordinamento globale completo.